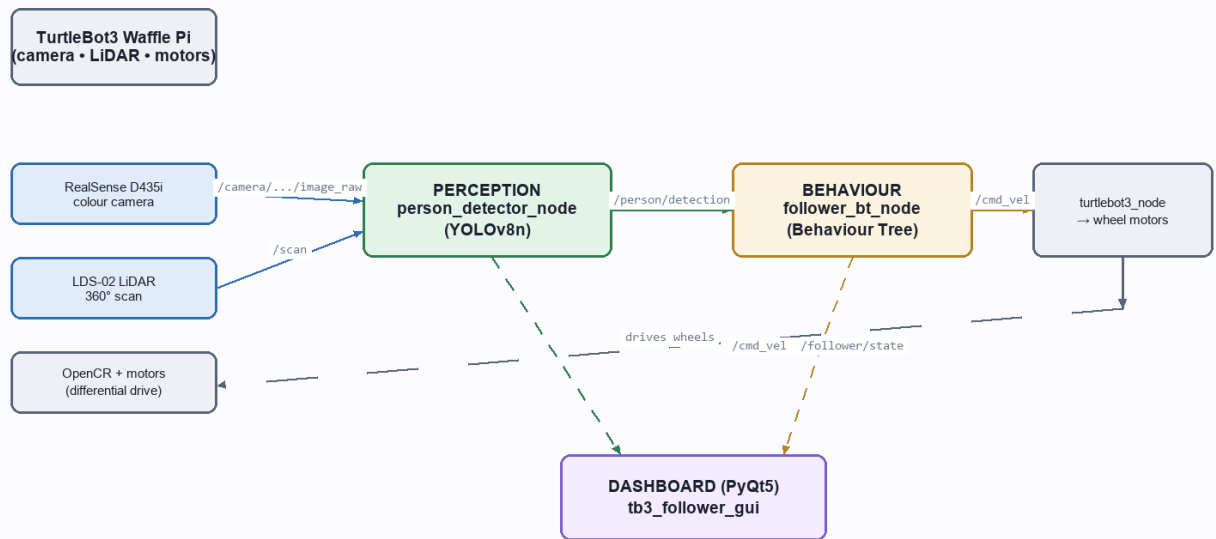


TurtleBot3 Human-Following Robot

YOLOv8 Perception · Behaviour Tree Control · LiDAR Obstacle Avoidance

ROS 2 Humble • Gazebo simulation • Real TurtleBot3 Waffle Pi

System Architecture — nodes (boxes) and ROS 2 topics (arrows)



A complete, from-scratch walkthrough: what the robot does, how every piece works, the code behind it, and the exact commands to run it in simulation and on real hardware.

Contents

1. Executive Summary — what it does
2. Big Picture — architecture & ROS 2 in 5 minutes
3. Hardware & Software Stack
4. Perception — seeing the person with YOLOv8
5. Estimating Distance — camera + LiDAR fusion
6. Behaviour — the decision-making Behaviour Tree
7. The Motion Controller — how fast, which way
8. LiDAR Obstacle Avoidance
9. Orient-to-Face
10. The Live Dashboard (GUI)
11. Networking — connecting the VM to the real robot
12. HOW TO RUN — simulation, step by step
13. HOW TO RUN — the real robot, step by step
14. One-Command Launch & the Dashboard buttons
15. Testing
16. Troubleshooting (the real problems we hit)
17. Command Cheat-Sheet
18. Appendix — key source files

1. Executive Summary — what it does

This project turns a TurtleBot3 into an autonomous human follower. A depth camera and an on-board neural network find the nearest person; a behaviour tree decides how to move; the robot drives toward the person, keeps a comfortable ~1 metre gap, turns to face them if they step to the side, and steers around obstacles using its LiDAR — all shown live on a desktop dashboard.

It runs in two modes from the same code base:

- **Simulation:** a Gazebo world with a 20×20 m room and three walking people; the robot follows the nearest one.
- **Real robot:** a physical TurtleBot3 Waffle Pi with an Intel RealSense camera and a 360° LiDAR.

Key capabilities:

- **Person detection:** YOLOv8 object detector, filtered to the 'person' class.
- **Nearest-target selection:** when several people are visible, it follows the closest.
- **Distance sensing:** a pinhole-camera estimate fused with the LiDAR for robustness.
- **Smooth following:** proportional control — approach when far, hold when in range, stop when too close.
- **Orient-to-face:** rotates in place to centre a person who is off to the side.
- **Obstacle avoidance:** reactive LiDAR avoidance that never mistakes the followed person for an obstacle.
- **Live GUI:** camera view with detection box, colour-coded state, and telemetry.

In one sentence: cameras + LiDAR feed a neural-network detector, whose output drives a priority-ordered behaviour tree that publishes wheel velocities — safe, legible, and testable.

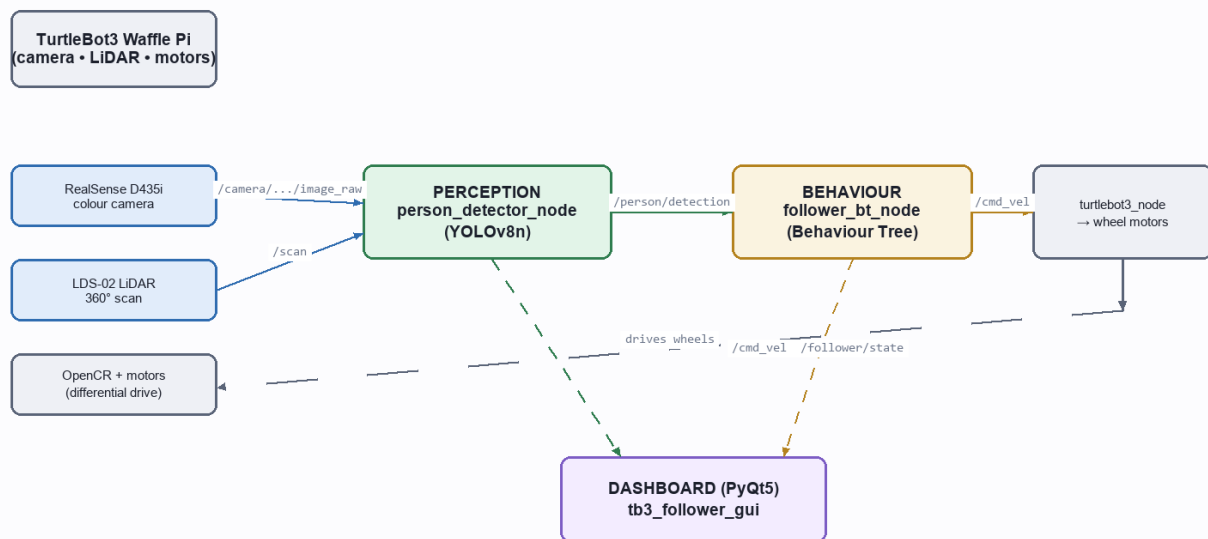
2. Big Picture — architecture & ROS 2 in 5 minutes

The three layers

The system is split into three cooperating programs (ROS 2 'nodes'), each with one job. This separation is what makes it easy to test, swap simulation for hardware, and debug.

- **Perception (person_detector_node):** reads the camera and LiDAR, runs YOLO, and publishes a single message describing the nearest person.
- **Behaviour (follower_bt_node):** reads that message plus the LiDAR, ticks a behaviour tree 10 times a second, and publishes a velocity command.
- **Dashboard (tb3_follower_gui):** a monitoring window; it only listens, never controls.

System Architecture — nodes (boxes) and ROS 2 topics (arrows)



Solid = data flow • Dashed = monitoring / actuation. Everything speaks ROS 2 (DDS) on ROS_DOMAIN_ID=30.

Figure 1 — Data flow. Boxes are nodes; arrows are ROS 2 topics.

ROS 2 concepts you need

Term	Meaning in this project
Node	One running program with a single responsibility (perception, behaviour, GUI).
Topic	A named message channel, e.g. <code>/person/detection</code> . Nodes publish and subscribe.
Message	A typed data packet on a topic (an image, a scan, a velocity, our <code>PersonDetection</code>).
Publish / Subscribe	Nodes are decoupled — a publisher doesn't know who listens. Add a logger by just subscribing; nothing else changes.

Parameter	A tunable value loaded at start-up (thresholds, gains, topic names) from a YAML file.
ROS_DOMAIN_ID	A number that isolates a ROS 2 network. VM and robot must share it (we use 30).

The main topics

Topic	Type	From → To
/camera/camera/color/image_raw	sensor_msgs/Image	Camera → Perception, Dashboard
/scan	sensor_msgs/LaserScan	LiDAR → Perception, Behaviour
/person/detection	tb3_follower_msgs/PersonDetection	Perception → Behaviour, Dashboard
/cmd_vel	geometry_msgs/Twist	Behaviour → Robot, Dashboard
/follower/state	std_msgs/String	Behaviour → Dashboard

3. Hardware & Software Stack

Real robot

Component	Part	Role
Base	TurtleBot3 Waffle Pi	Differential-drive mobile base.
Computer	Raspberry Pi 4	Runs the robot drivers + camera node (ROS 2 Humble).
Controller	OpenCR board	Motor control, IMU, battery; talks to the Pi over USB.
Camera	Intel RealSense D435i	Colour images for person detection.
LiDAR	LDS-02	360° laser scan for distance + obstacle avoidance.

Development machine

Layer	Choice	Notes
Host OS	Windows 11	Runs VirtualBox and the phone-style Mobile Hotspot the robot joins.
Guest OS	Ubuntu 22.04 in VirtualBox	The 'Remote PC' that runs perception + behaviour + GUI.
Middleware	ROS 2 Humble	Same version on the VM and the Pi (required for DDS compatibility).
Perception	Ultralytics YOLOv8n	Nano model — small and fast enough for CPU.
Behaviour	py_trees	Python behaviour-tree library.
Simulator	Gazebo Classic	Physics world for testing without hardware.
GUI	PyQt5 + OpenCV	The dashboard.

Why a VM? It keeps a clean, reproducible Ubuntu/ROS 2 environment on a Windows laptop and makes the exact same code run in simulation and against the real robot.

4. Perception — seeing the person with YOLOv8

The perception node subscribes to the colour image, the camera calibration, and the LiDAR. For each frame it runs YOLOv8 restricted to the COCO 'person' class, computes a distance for every detected person, and publishes just the nearest one.

The published message

A compact, custom message keeps the interface between perception and behaviour tiny and typed:

```
# PersonDetection.msg (tb3_follower_msgs)
bool    detected          # was a person found this frame?
float32 bbox_cx           # box centre X, normalised 0..1 (0.5 = dead ahead)
float32 bbox_cy           # box centre Y, normalised 0..1
float32 bbox_w            # box width, normalised 0..1
float32 bbox_h            # box height, normalised 0..1
float32 distance          # metres to the person (-1 if unknown)
float32 confidence        # detector confidence 0..1
builtin_interfaces/Time stamp
```

The detection loop (nearest-person)

Rather than blindly trusting YOLO's most-confident box, we score every box by distance and follow the closest. This is the core of multi-person handling:

```
# person_detector_node.py (core of _process_frame)
results = self.model.predict(source=frame, imgsiz=self.imgsz,
                             conf=self.conf_threshold, classes=[0], verbose=False)

boxes = results[0].boxes
confs = boxes.conf.cpu().numpy()
xyxy = boxes.xyxy.cpu().numpy()

candidates = []
for i in range(len(confs)):
    x1, y1, x2, y2 = xyxy[i].tolist()
    bbox_h_px = max(y2 - y1, 1.0)
    bbox_cx = float(((x1 + x2) / 2.0) / w_px) # normalise to 0..1

    d_visual = visual_distance(person_height_m=self.person_height_m,
                               bbox_h_px=bbox_h_px, focal_px=self.focal_px)
    d_lidar = float("nan")
    if self.last_scan is not None:
        idx = bbox_cx_to_lidar_index(bbox_cx=bbox_cx, camera_hfov_rad=self.camera_hfov_rad,
                                     scan_angle_min=self.last_scan.angle_min,
                                     scan_angle_increment=self.last_scan.angle_increment,
                                     num_beams=len(self.last_scan.ranges))
        if 0 <= idx < len(self.last_scan.ranges):
            d_lidar = float(self.last_scan.ranges[idx])

    distance = fuse_distance(visual_d=d_visual, lidar_d=d_lidar,
                            lidar_range_max=self.last_scan.range_max)
    candidates.append({"bbox_cx": bbox_cx, "distance": distance, ...})

best = candidates[nearest_index([c["distance"] for c in candidates])] # follow the closest
```

`nearest_index()` is a tiny, pure function — it ignores invalid readings (NaN, ∞ , ≤ 0) and returns the index of the smallest valid distance, so a bad reading can never win:

```
def nearest_index(distances):
    best_i, best_d = -1, float("inf")
    for i, d in enumerate(distances):
        if math.isnan(d) or math.isinf(d) or d <= 0:
            continue
        if d < best_d:
            best_i, best_d = i, d
    return best_i if best_i >= 0 else 0
```

Performance trick: the node rate-limits to ~10 Hz and drops a frame if inference is still running (a 'busy' flag), so a slow CPU never builds a backlog of stale images.

5. Estimating Distance — camera + LiDAR fusion

Knowing how far the person is drives the whole follow behaviour. We compute it two ways and combine them.

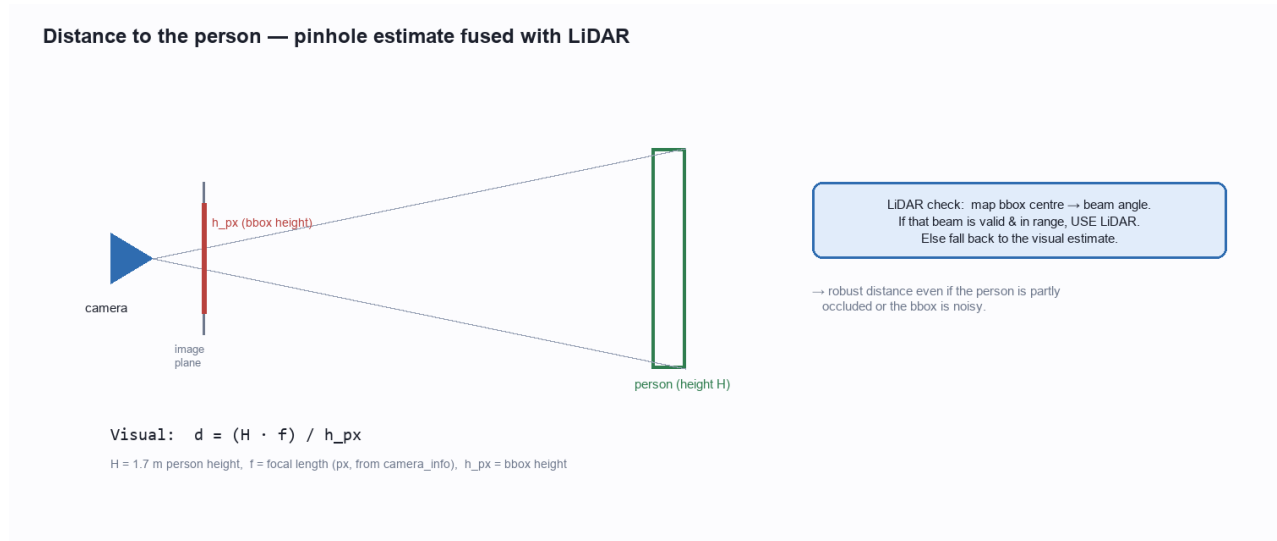


Figure 2 — Pinhole distance from bounding-box height, checked against the LiDAR.

Visual estimate (pinhole model)

A camera is a pinhole: a real object of height H metres projects to h_{px} pixels tall, and by similar triangles the distance is $d = (H \cdot f) / h_{px}$, where f is the focal length in pixels (read once from the camera's /camera_info). Assuming a 1.7 m person, a shorter box means the person is farther away.

```
def visual_distance(*, person_height_m, bbox_h_px, focal_px):
    if bbox_h_px <= 0 or focal_px <= 0:
        raise ValueError("must be > 0")
    return (person_height_m * focal_px) / bbox_h_px          # d = H*f / h_px
```

LiDAR cross-check and fusion

The bounding box tells us the person's horizontal angle; we map that angle to the matching LiDAR beam and read the true range there. If that beam is valid and in range we trust the LiDAR (accurate); otherwise we fall back to the visual estimate (always available).

```
def fuse_distance(*, visual_d, lidar_d, lidar_range_max):
    if math.isnan(lidar_d) or math.isinf(lidar_d): return visual_d
    if lidar_d <= 0 or lidar_d >= lidar_range_max: return visual_d
    return lidar_d                                     # LiDAR wins when trustworthy
```

The angle→beam mapping keeps the camera and LiDAR reference frames consistent:

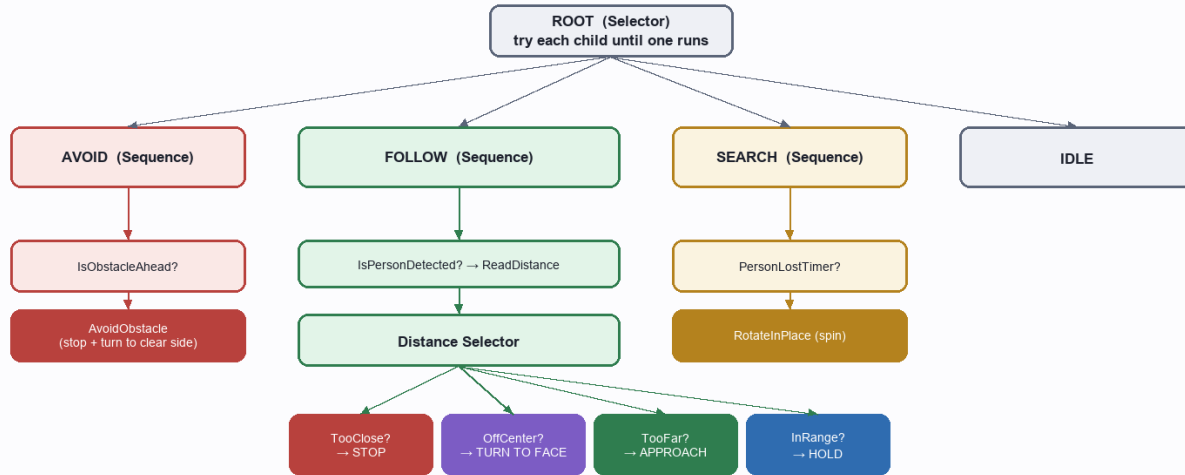
```
def bbox_cx_to_lidar_index(*, bbox_cx, camera_hfov_rad, scan_angle_min,
                           scan_angle_increment, num_beams):
    yaw = (0.5 - bbox_cx) * camera_hfov_rad          # 0.5 -> straight ahead
    raw = (yaw - scan_angle_min) / scan_angle_increment
    return int(round(raw)) % num_beams
```

6. Behaviour — the decision-making Behaviour Tree

A behaviour tree (BT) is a clean way to express robot decisions. It is 'ticked' 10 times a second; each tick flows from the top and asks each branch, in priority order, whether it wants to run.

- **Selector (?):** try children left-to-right; stop at the first that succeeds or runs. Used for priority.
- **Sequence (→):** run children in order; all must succeed. Used for 'condition THEN action'.
- **Status:** each node returns SUCCESS, FAILURE, or RUNNING every tick.

The Follower Behaviour Tree (ticked at 10 Hz, top-to-bottom priority)



Priority order: safety (AVOID) → follow the person → search if lost → idle.

Each tick the tree publishes its active leaf on `/follower/state`, which colours the dashboard badge.

Figure 3 — The follower behaviour tree. Left-to-right is highest-to-lowest priority.

Reading the tree, top to bottom

- **AVOID (highest priority):** if the LiDAR sees a non-person obstacle close ahead, stop and steer around it — safety overrides everything.
- **FOLLOW:** if a person is detected, pick one of four actions by distance and angle: STOP (too close), TURN-TO-FACE (off to the side), APPROACH (too far), or HOLD (in range).
- **SEARCH:** if the person was lost for longer than a timeout, rotate in place to look for them.
- **IDLE:** a safety net — publish zero velocity if nothing else applies.

How the tree is built (code)

```
# tree.py – build_tree() (abridged)
distance_selector = Selector("DistanceSelector", children=[
    Sequence("TooClose->Stop", [DistanceWithin(hi=close_threshold), Stop(...)]),
    Sequence("OffCenter->Turn", [IsOffCenter(orient_offset_threshold), TurnToFace(...)]),
    Sequence("TooFar->Approach", [DistanceWithin(lo=far_threshold), Approach(...)]),
    Sequence("InRange->Hold", [DistanceWithin(close, far), HoldPosition(...)]),
])
follow = Sequence("FOLLOW", [IsPersonDetected(timeout), ReadDistance(), distance_selector])
```

```
avoid = Sequence("AVOID", [IsObstacleAhead(obstacle_params), AvoidObstacle(...)])
search = Sequence("SEARCH", [PersonLostTimer(timeout), RotateInPlace(yaw_rate)])
root = Selector("Root", [avoid, follow, search, Idle(...)]) # priority order
```

Why a tree and not if/else? priorities are explicit and visual, each condition/action is a small unit you can unit-test in isolation, and the currently-running leaf is exactly the robot's 'state' — which we publish for the dashboard.

7. The Motion Controller — how fast, which way

The APPROACH action turns a distance and a bounding-box offset into a wheel command using simple proportional control. No machine learning here — just clear, tunable maths.

Follow control — what the robot does at each distance

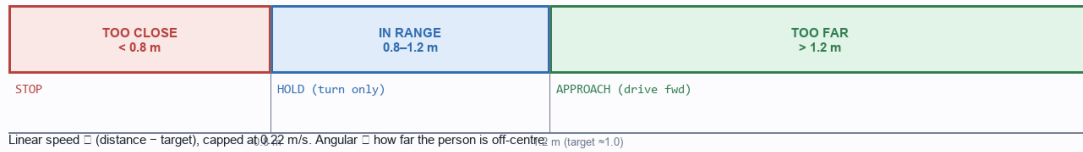


Figure 4 — Distance zones decide the action; speeds are proportional.

```
def compute_approach_twist(*, distance, bbox_cx, params):
    # forward speed grows with how far past the target the person is (never negative)
    lin_err = distance - params.target_distance
    v = clamp(params.k_linear * lin_err, 0.0, params.max_linear_speed)
    # turn toward the person: bbox_cx < 0.5 (left of frame) -> positive (left) turn
    ang_err = -(bbox_cx - 0.5)
    w = clamp(params.k_angular * ang_err, -params.max_angular_speed, params.max_angular_speed)
    return v, w
```

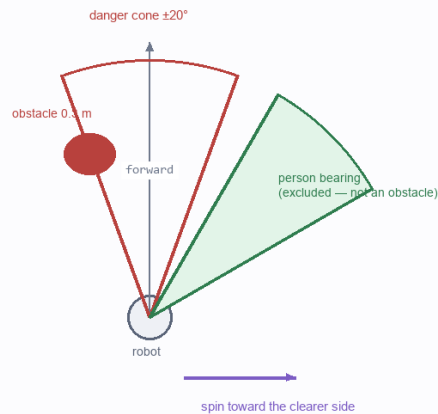
Defaults (all tunable in follower_params.yaml):

Parameter	Value	Meaning
target_distance	1.0 m	Gap the robot tries to keep.
close_threshold	0.8 m	Below this → STOP.
far_threshold	1.2 m	Above this → APPROACH.
max_linear_speed	0.22 m/s	TB3 top speed.
max_angular_speed	1.0 rad/s	Turn-rate cap.
k_linear / k_angular	0.4 / 1.5	Proportional gains (lower = gentler).
orient_offset_threshold	0.22	How far off-centre (0..0.5) before turning in place.

8. LiDAR Obstacle Avoidance

Following a person is no good if the robot drives into a wall. The AVOID branch adds reactive safety using the 360° LiDAR — with one essential subtlety: it must not treat the person it is following as an obstacle.

LiDAR obstacle avoidance — frontal cone, person excluded, turn to clear side



Rule (assess_obstacle):

1. Look only at beams within $\pm$ front_half of straight ahead.
2. Ignore beams within person_margin of the person's bearing (that return IS the person).
3. If the nearest remaining return < stop_distance (0.4 m):
→ present = True (an obstacle is in the way).
4. Compare clearance left vs right → turn_sign.
5. AvoidObstacle: linear = 0, angular = turn_sign · rate.

Figure 5 — Frontal danger cone, the person's bearing excluded, and the escape turn.

The logic

- Look only at LiDAR beams within a frontal cone ($\pm 20^\circ$) — obstacles to the side don't matter while driving forward.
- Ignore beams within a small margin of the person's known bearing (from the bounding box) — that return is the person.
- If the nearest remaining return is closer than 0.4 m, an obstacle is in the way.
- Compare clearance on the left vs the right and turn toward the more open side; stop forward motion while turning.

It's a pure, unit-tested function — no ROS types, so it runs anywhere:

```
def assess_obstacle(*, ranges, angle_min, angle_increment, range_max,
                    front_half_rad, stop_distance,
                    person_bearing_rad=None, person_margin_rad=0.0, side_half_rad=0.9):
    front_min = float("inf")
    for i, r in enumerate(ranges):
        theta = _norm(angle_min + i * angle_increment)          # wrap to [-pi, pi]
        if abs(theta) > front_half_rad:                          continue # not ahead
        if person_bearing_rad is not None and \
            abs(_norm(theta - person_bearing_rad)) <= person_margin_rad: continue # the person
        if _valid(r, range_max) and r < front_min:              front_min = r
    # choose escape side from left/right clearance (excluding the central cone)
    left = sector_min_range(ranges, ..., +side_center, side_hw, range_max)
    right = sector_min_range(ranges, ..., -side_center, side_hw, range_max)
    return ObstacleAssessment(present=front_min < stop_distance, front_min=front_min,
                              turn_sign=1.0 if left >= right else -1.0)
```

The behaviour-tree guard and action are thin wrappers around it:

```
class IsObstacleAhead(Behaviour):          # returns SUCCESS if an obstacle is present
    def update(self):
        person_bearing = (0.5 - det.bbox_cx) * hfov if det and det.detected else None
        a = assess_obstacle(ranges=scan.ranges, ..., person_bearing_rad=person_bearing)
        self.bb.set(KEY_AVOID, a)
        return SUCCESS if a.present else FAILURE

class AvoidObstacle(Behaviour):            # stop + spin toward the clear side
    def update(self):
        a = self.bb.get(KEY_AVOID)
        self.twist_pub.send(0.0, a.turn_sign * self.params.avoid_yaw_rate)
        return RUNNING
```

9. Orient-to-Face

If a detected person is far off to one side, driving forward would send the robot off at an angle. Instead it first rotates in place to centre them, then approaches — smoother and more natural.

```
class IsOffCenter(Behaviour):    # person more than `threshold` off centre?
    def update(self):
        if not (det and det.detected): return FAILURE
        return SUCCESS if abs(det.bbox_cx - 0.5) > self.threshold else FAILURE

class TurnToFace(Behaviour):    # rotate in place toward the person (no forward motion)
    def update(self):
        _, w = compute_approach_twist(distance=target, bbox_cx=det.bbox_cx, params=params)
        self.twist_pub.send(0.0, w)
        return RUNNING
```

It sits in the FOLLOW selector just after the safety STOP, so the order of preference is: too-close → STOP, off-centre → TURN-TO-FACE, too-far → APPROACH, otherwise → HOLD.

10. The Live Dashboard (GUI)

The PyQt5 dashboard is a read-only window that makes the robot's 'thinking' visible — invaluable for demos and debugging. It shows the camera with the detection box drawn on top, a big colour-coded state badge, and telemetry cards (distance, confidence, box centre, wheel speeds, and message rates).

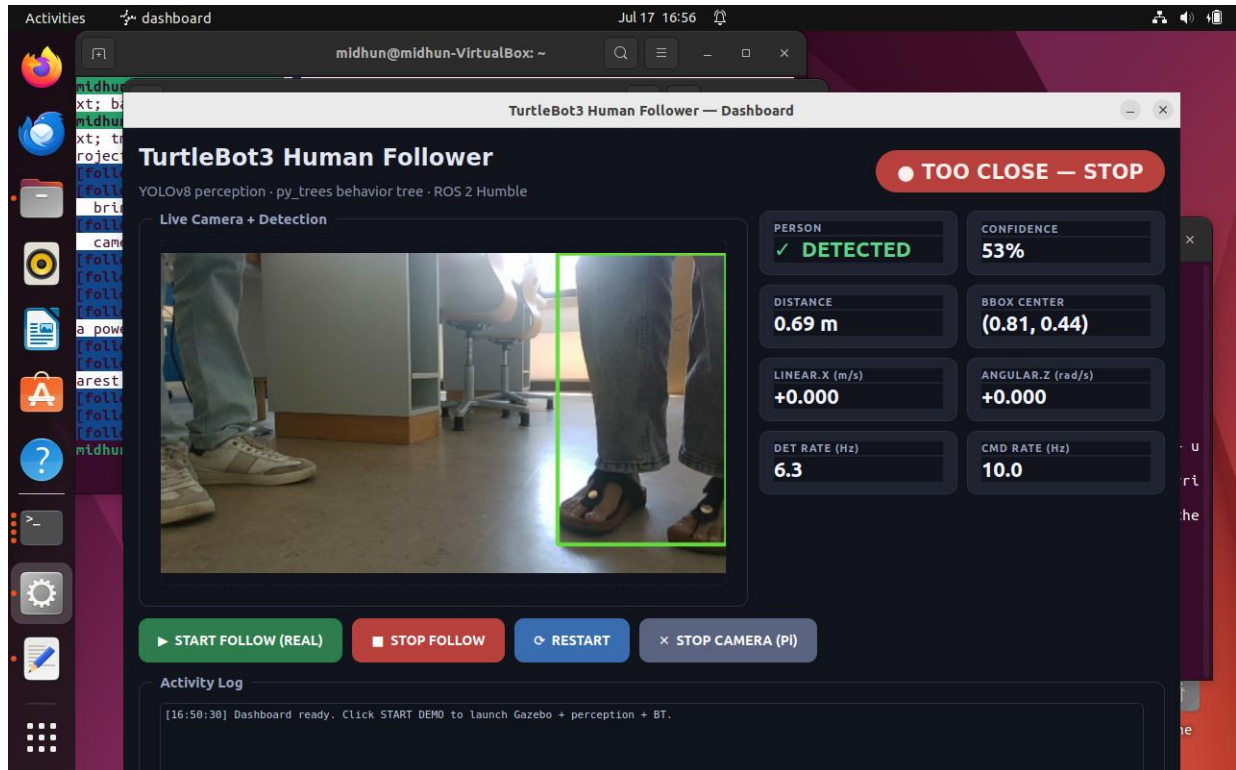


Figure 6 — The dashboard: live camera + green detection box, state badge, and telemetry.

Showing the real state

Rather than guessing the state from distance, the behaviour node publishes its actual active branch on `/follower/state` every tick, and the dashboard simply displays it — so ORIENTING and AVOIDING show correctly, not just approach/hold/stop.

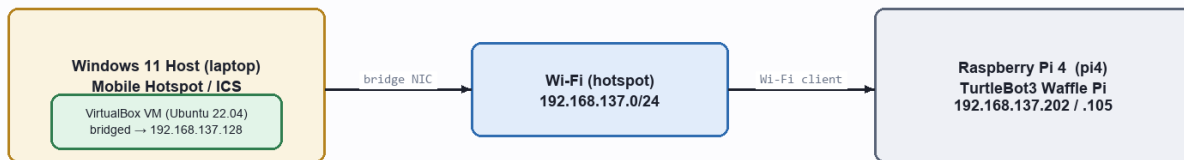
```
# follower_bt_node.py — publish the active leaf each tick
_TIP_TO_STATE = {"AvoidObstacle": "avoid", "TurnToFace": "orient", "Approach": "approach",
                 "HoldPosition": "hold", "Stop": "stop", "RotateInPlace": "searching", "Idle": "stopped"}
def _publish_state(self):
    tip = self.root.tip() # the currently-running leaf
    msg = String(); msg.data = _TIP_TO_STATE.get(tip.name if tip else "", "stopped")
    self.state_pub.publish(msg)
```

The START button on the dashboard launches the whole real-robot follower; STOP stops it. State badges: SEARCHING, APPROACHING, HOLDING, TOO CLOSE — STOP, ORIENTING, and AVOIDING OBSTACLE.

11. Networking — connecting the VM to the real robot

Getting the VM and the robot to talk was the trickiest real-world step. The robot's Raspberry Pi joins the Windows laptop's Mobile Hotspot; the VM is bridged onto that same hotspot so both sit on one 192.168.137.x network and ROS 2 discovers across it.

Networking — Windows Mobile Hotspot bridges the VM to the real robot



- Both sides set `ROS_DOMAIN_ID=30` and `ROS_LOCALHOST_ONLY=0` so DDS discovers across the LAN.
- VM NIC is bridged to 'Microsoft Wi-Fi Direct Virtual Adapter #2' (the hotspot), NOT the laptop Wi-Fi.
- From the host you can reach the Pi directly: `plink -ssh -pw turtlebot ubuntu@192.168.137.105`

Figure 7 — The VM is bridged onto the Windows hotspot the robot joins.

- **Bridge the VM:** attach the VM NIC to 'Microsoft Wi-Fi Direct Virtual Adapter #2' (the hotspot), not the laptop's own Wi-Fi. The VM then gets a 192.168.137.x address by DHCP.
- **Match the domain:** export `ROS_DOMAIN_ID=30` and `ROS_LOCALHOST_ONLY=0` on BOTH the VM and the Pi.
- **Verify:** from the VM, ping the Pi and run `ros2 topic list` — you should see `/scan`, `/odom`, `/cmd_vel`.

```
# On the Windows host, hot-swap the VM NIC to the hotspot (no reboot):
VBoxManage controlvm ros2-tb3-follower nic1 bridged "Microsoft Wi-Fi Direct Virtual Adapter #2"
# From the host you can even SSH straight into the Pi:
plink -ssh -pw turtlebot ubuntu@192.168.137.105 "hostname"
```

Gotcha we hit: the VM's `~/.bashrc` exported `ROS_DOMAIN_ID=7`, so scripts must force domain 30 explicitly — otherwise the follower and the robot are on different domains and never see each other.

12. HOW TO RUN — simulation, step by step

The simulation needs no hardware. Every VM terminal first sources ROS 2 and the workspace:

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
```

Option A — one command (3-pane tmux)

```
bash /mnt/host_project/scripts/05-run-demo.sh
```

This opens Gazebo + the YOLO detector + the behaviour tree together. Gazebo shows a 20×20 m room with three walking people; the robot follows the nearest.

Option B — three terminals (to see each part)

```
export TURTLEBOT3_MODEL=waffle_pi
export DISPLAY=:0
export GAZEBO_MODEL_PATH=$GAZEBO_MODEL_PATH:/opt/ros/humble/share/turtlebot3_gazebo/models

# Terminal 1 – Gazebo world + robot
ros2 launch tb3_follower_bringup sim.launch.py
# Terminal 2 – YOLO person detector
ros2 launch tb3_follower_bringup perception.launch.py
# Terminal 3 – behaviour-tree follower
ros2 launch tb3_follower_bringup behavior.launch.py
```

Watch it work

```
ros2 topic echo /person/detection    # detected? distance to nearest person
ros2 topic echo /cmd_vel              # the velocity commands driving the robot
ros2 topic hz /person/detection      # detection rate (~10 Hz)
```

Stop the simulation:

```
tmux kill-session -t tb3-follower
```

13. HOW TO RUN — the real robot, step by step

Step 1 — power on & network

- Turn on the TurtleBot3; wait ~1–2 min for the Pi to boot and join the Windows hotspot.
- Confirm the VM is bridged to the hotspot adapter and can ping the Pi (192.168.137.105).

Step 2 — on the Pi: start the robot drivers + camera

```
ssh ubuntu@192.168.137.105          # password: turtlebot
export ROS_DOMAIN_ID=30 TURTLEBOT3_MODEL=waffle_pi LDS_MODEL=LDS-02
# robot base (motors, IMU, LiDAR, odometry):
ros2 launch turtlebot3_bringup robot.launch.py
# RealSense colour stream (in another Pi shell), low-res for smooth Wi-Fi:
ros2 launch realsense2_camera rs_launch.py \
    enable_depth:=false pointcloud.enable:=false rgb_camera.color_profile:=424x240x15
```

Step 3 — on the VM: run the follower

```
export ROS_DOMAIN_ID=30 ROS_LOCALHOST_ONLY=0 TURTLEBOT3_MODEL=waffle_pi
# YOLO detector, pointed at the RealSense colour topic:
ros2 run tb3_follower_perception person_detector_node --ros-args \
    --params-file ~/ros2_ws/src/tb3_follower_perception/config/detector_params.yaml \
    -p image_topic:=/camera/camera/color/image_raw \
    -p camera_info_topic:=/camera/camera/color/camera_info
# behaviour tree (in another VM shell) – publishes /cmd_vel to the real robot:
ros2 launch tb3_follower_bringup behavior.launch.py
```

Safety: the real robot MOVES. Give it a few metres of clear floor and keep the STOP command (``tmux kill-session -t tb3-follow``) or the Pi power switch handy.

Step 4 — stop everything

```
# VM side:
tmux kill-session -t tb3-follow
# Robot side (halt motors + drivers):
sshpass -p turtlebot ssh ubuntu@192.168.137.105 \
    "pkill -f robot.launch; pkill -f realsense2_camera_node"
```

14. One-Command Launch & the Dashboard buttons

A single script automates the whole real-robot bring-up: it SSHes to the Pi and starts the drivers + camera if needed, waits for the topics to appear, then launches the YOLO detector and behaviour tree on the VM.

```
bash /mnt/host_project/scripts/06-follow-real.sh
# override the Pi password if different:
PI_PASS=yourpass bash /mnt/host_project/scripts/06-follow-real.sh
# watch / stop:
tmux attach -t tb3-follow      # live view (detach: Ctrl+B then D)
tmux kill-session -t tb3-follow # stop the follower
```

The dashboard wraps this in buttons: START FOLLOW (REAL) runs the script, STOP FOLLOW stops it, and STOP CAMERA (Pi) shuts the RealSense down. Launch the dashboard with:

```
ros2 run tb3_follower_gui dashboard --ros-args \
  -r /camera/image_raw:=/camera/camera/color/image_raw
```

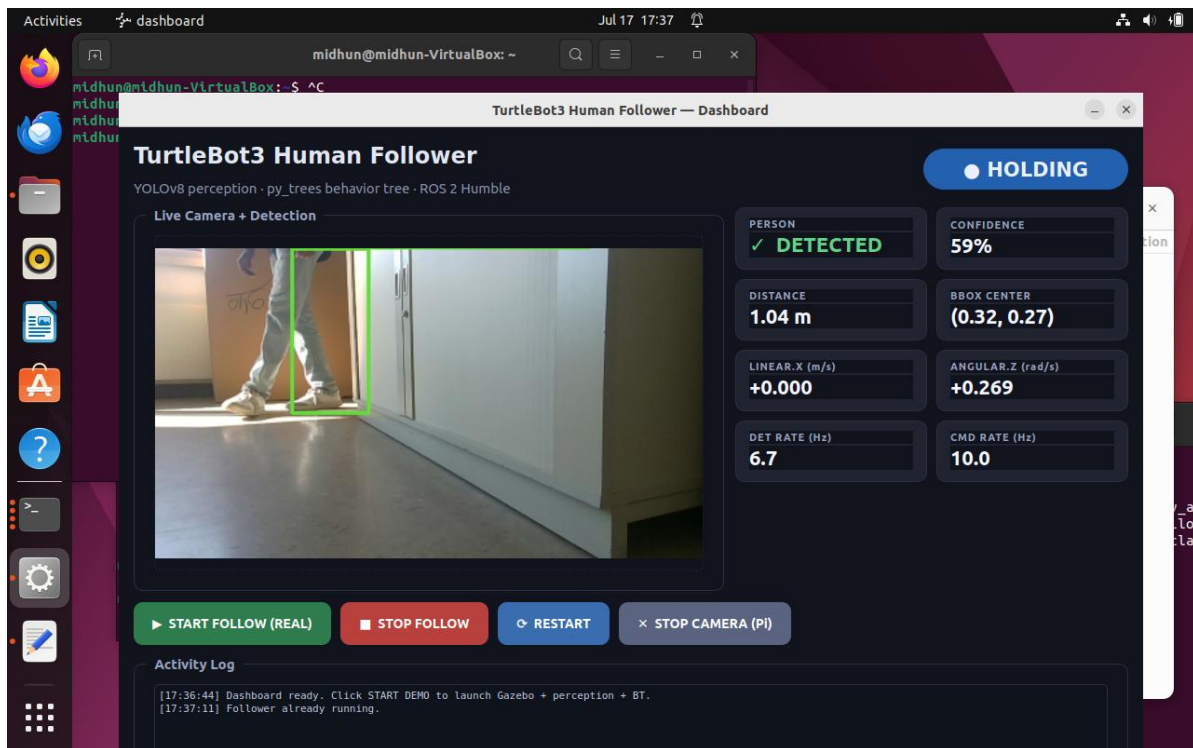


Figure 8 — Dashboard in HOLDING state driven live from /follower/state.

15. Testing

The maths lives in pure, ROS-free functions so it can be unit-tested on any machine — fast feedback without a robot or simulator.

```
cd ~/ros2_ws/src/tb3_follower_perception && python3 -m pytest test/ -q
cd ~/ros2_ws/src/tb3_follower_behavior && python3 -m pytest test/ -q
```

Test file	Covers
test_geometry.py	visual distance, angle→beam mapping, LiDAR fusion, nearest-person selection.
test_control.py	proportional approach controller (speed & turn, clamping, direction).
test_obstacle.py	front-cone obstacle detection, person exclusion, left/right escape choice.
test_tree_ticks.py	the full tree: search, approach, hold, stop, orient, avoid, disabled-avoid.

Roughly 40 tests cover the decision logic end to end, including edge cases like ties, invalid readings, and 'the person is the closest thing in front but must not be avoided'.

16. Troubleshooting (the real problems we hit)

Symptom	Cause	Fix
VM window crashes on start ('memory could not be read')	8 GB VM on a 16 GB host — the GUI can't commit that much.	Run the VM at 6 GB (or 4 GB headless).
RealSense: 'UVC control ... timed out', no image	Raspberry Pi under-voltage — camera browns out.	Charge the battery / use a powered USB-3 hub; check `vcgencmd get_throttled` reads 0x0.
Follower detects a person but only rotates	Behaviour dropped to SEARCH on every missed frame; stale detection over a slow link.	Latch only positive detections; raise person_lost_timeout to 2.5 s.
Robot & follower can't see each other	Different ROS_DOMAIN_ID (VM bashrc set 7, robot uses 30).	Force ROS_DOMAIN_ID=30 on both; ROS_LOCALHOST_ONLY=0.
Video laggy over Wi-Fi	640×480 raw is too much bandwidth.	Stream 424×240; optional compressed transport.
Can't reach the Pi from the VM	VM on NAT, or bridged to the wrong adapter.	Bridge to the Wi-Fi Direct (hotspot) adapter; DHCP gives a 192.168.137.x IP.

17. Command Cheat-Sheet

Every terminal (VM)

```
source /opt/ros/humble/setup.bash && source ~/ros2_ws/install/setup.bash
```

Simulation

```
bash /mnt/host_project/scripts/05-run-demo.sh      # start
tmux kill-session -t tb3-follower                  # stop
```

Real robot

```
bash /mnt/host_project/scripts/06-follow-real.sh    # start (auto Pi bring-up + follower)
tmux kill-session -t tb3-follow                     # stop follower
sshpass -p turtlebot ssh ubuntu@192.168.137.105 "pkill -f robot.launch; pkill -f realsense2_camera_node"
```

Inspect

```
ros2 node list
ros2 topic list
ros2 topic echo /person/detection
ros2 topic echo /cmd_vel
ros2 topic echo /follower/state
ros2 topic hz /camera/camera/color/image_raw
```

Tune (edit, then restart the behaviour node)

```
nano ~/ros2_ws/src/tb3_follower_behavior/config/follower_params.yaml
# target_distance, close/far_threshold, k_linear, k_angular,
# obstacle_stop_distance, avoid_yaw_rate, orient_offset_threshold
```

18. Appendix — key source files

Full behaviour-tree assembly (tree.py):

```
def build_tree(*, twist_pub, params, obstacle_params, person_lost_timeout,
               search_yaw_rate, orient_offset_threshold=0.22):
    distance_selector = Selector("DistanceSelector", memory=False, children=[
        Sequence("TooClose->Stop", memory=False, children=[
            DistanceWithin("IsTooClose", lo=-inf, hi=params.close_threshold), Stop(twist_pub)]),
        Sequence("OffCenter->TurnToFace", memory=False, children=[
            IsOffCenter(threshold=orient_offset_threshold), TurnToFace(twist_pub, params)]),
        Sequence("TooFar->Approach", memory=False, children=[
            DistanceWithin("IsTooFar", lo=params.far_threshold, hi=inf), Approach(twist_pub, params)]),
        Sequence("InRange->Hold", memory=False, children=[
            DistanceWithin("InRange", lo=params.close_threshold, hi=params.far_threshold),
            HoldPosition(twist_pub, params)]),
    ])
    follow = Sequence("FOLLOW", memory=False, children=[
        IsPersonDetected(timeout_s=person_lost_timeout), ReadDistance(), distance_selector])
    avoid = Sequence("AVOID", memory=False, children=[
        IsObstacleAhead(params=obstacle_params), AvoidObstacle(twist_pub, obstacle_params)])
    search = Sequence("SEARCH", memory=False, children=[
        PersonLostTimer(timeout_s=person_lost_timeout), RotateInPlace(twist_pub, search_yaw_rate)])
    return Selector("Root", memory=False, children=[avoid, follow, search, Idle(twist_pub)])
```

Behaviour parameters (follower_params.yaml):

```
follower_bt_node:
  ros_parameters:
    target_distance: 1.0
    close_threshold: 0.8
    far_threshold: 1.2
    max_linear_speed: 0.22
    max_angular_speed: 1.0
    k_linear: 0.4
    k_angular: 1.5
    person_lost_timeout: 2.5
    search_yaw_rate: 0.3
    tick_rate_hz: 10.0
    # obstacle avoidance
    obstacle_avoidance_enabled: true
    obstacle_stop_distance: 0.4
    obstacle_front_half_rad: 0.35
    obstacle_person_margin_rad: 0.26
    avoid_yaw_rate: 0.6
    camera_hfov_rad: 1.085
    scan_topic: "/scan"
    # orient-to-face + state
    orient_offset_threshold: 0.22
    follower_state_topic: "/follower/state"
```

Perception parameters (detector_params.yaml):

```
person_detector_node:
  ros_parameters:
    model_path: "~/models/yolov8n.pt"
    conf_threshold: 0.5
    imgsz: 320
    max_rate_hz: 10.0
    person_height_m: 1.7
    camera_hfov_rad: 1.085
    image_topic: "/camera/image_raw"
```



```
camera_info_topic: "/camera/camera_info"  
scan_topic: "/scan"  
detection_topic: "/person/detection"
```

— *End of tutorial* —